

แบบทดสอบภาคปฏิบัติ: ระบบ Spring Boot - Customer-Order-OrderLine-Product

ชื่อ: ยศนพร งามสง รหัสนักศึกษา: 663390511-8

วันที่: 10/11/68 คะแนนรวม: 120 คะแนน

ตอนที่ 1 Implementation (50 คะแนน)

ส่วนที่ 1: จงพัฒนา Restfull Api เพื่อการอ่าน ค้นหา บันทึก เพิ่ม ปรับปรุง ลบ ข้อมูลของใบ order ซึ่งสัมพันธ์กับ Customer, OrderLine, Product ด้วย spring framework ส่ง เป็น ไฟล์ zip ของโปรเจ็ค (30 คะแนน)

ส่วนที่ 2 ให้แสดงการทดสอบ โดยใช้ PostMan ผ่านคลิปลิ้น ๆ ให้ครบทุก url การค้นหาทั้งหมด ค้นหาโดยรหัส เพิ่มใบคำสั่งซื้อใหม่ ปรับปรุง ลบ (10 คะแนน)

ส่วนที่ 3 ให้อธิบายคำสั่งในส่วนต่าง ๆ ที่สำคัญโดยเฉพาะ Service และ Controller โดยให้เห็นหน้าผู้อธิบายด้วย บันทึกลงคลิปลิ้น ๆ ไม่เกิน 5 นาที (10 คะแนน)

ตอนที่ 2 ตอบคำถามให้เขียนด้วยลายมือ และส่งเป็นไฟล์ pdf

ส่วนที่ 1: คำถามเชิงแนวคิด (20 คะแนน)

1. จงอธิบายความสัมพันธ์ระหว่างเอนทิตี Customer, Order, OrderLine และ Product ในระบบ Spring Boot

ตอบ: Customer (1→M) / Order (1→M) / OrderLine (M→1) / Product (M→1)

2. จงอธิบายความแตกต่างระหว่าง @OneToMany และ @ManyToOne ใน JPA คืออะไร? ยกตัวอย่างจากระบบของคุณ

ตอบ: OneToMany = หนึ่ง entity หนึ่ง entity

ManyToOne = หลาย entity หนึ่ง entity

3. จงอธิบายว่าเหตุใดจึงใช้ @JoinColumn(name = "customer_id") ในคลาส Order และจะเกิดอะไรขึ้นหากไม่

ใช้คำสั่งนี้

ตอบ: เพื่อสร้าง foreignkey ของ customer_id ใน Order ถ้าไม่กำหนด จะเผลอชื่อที่ตัวอัตโนมัติ

4. จงอธิบายว่าใครควรใช้ CascadeType.ALL ในความสัมพันธ์ของเอนทิตี และมีความเสี่ยงอย่างไร

คำตอบ: ใช้ในกรณีที่ ถ้าผมลบความสัมพันธ์ไปต้องลบคุณค่า ถ้าเอนทิตี
ลบแล้ว คืออาจลบข้อมูลโดยไม่ได้ตั้งใจ จาก Order ถูกลบ ส่งผลถึง Orderline

5. ทำไมคลาส Order จึงใช้ @Table(name = "orders")? และช่วยแก้ปัญหาอะไร

คำตอบ: เพราะ Order เป็นที่สงวน กับ มีชื่อ syntax error

ส่วนที่ 2: ความเข้าใจเกี่ยวกับ JSON และ API (20 คะแนน)

6. จงเขียน JSON เพื่อสร้าง Order ใหม่ที่เชื่อมกับ Customer (ID = 3) และมีสินค้า 2 รายการ (ID = 1 และ 2)
โดยแต่ละสินค้าระบุจำนวนและราคา

คำตอบ: _____
{
 "customerId": 3,
 "orderlines": [
 {
 "productId": 1,
 "quantity": 2,
 "price": 100.0
 },
 {
 "productId": 2,
 "quantity": 1,
 "price": 200.0
 }
]
}

7. เมื่อส่ง JSON ข้างต้น Spring Boot จะทำการแมปข้อมูล product และ customer แบบซ้อน (nested) ไปยังอ็อบเจกต์ Entity ได้อย่างไร

คำตอบ: spring boot ใช้ Jackson ในการแปลง json เป็น object

8. หาก API ตอบกลับด้วย "400 Bad Request" ระบุสาเหตุที่อาจเกิดขึ้นได้อย่างน้อย 3 ข้อ

คำตอบ: _____

1. ข้อมูลที่ส่งไปไม่ถูกต้อง ตามรูปแบบที่กำหนด
2. ข้อมูลไม่ครบหรือมีค่า null
3. ค่า parameter ไม่ถูกต้อง

9. จง แสดงตัวอย่าง JSON ที่ได้จากการเรียกใช้ GET /api/orders/1 และอธิบายส่วนประกอบของผลลัพธ์แต่ละส่วน

คำตอบ: _____

```
{
  "id": "cool",
  "name": "Yasin",
  "email": "yasin@example.com"
}
```

ส่วนที่ 3: การเขียนโค้ดและการแก้ไขปัญหา (15 คะแนน)

10. พิจารณาโค้ดต่อไปนี้ในคลาส OrderController:

```
@PostMapping
public Order createOrder(@RequestBody Order order) {
    return orderRepository.save(order);
}
```

จงอธิบายเหตุผลว่าโค้ดนี้อาจล้มเหลวหาก customer หรือ product ใน order ยังไม่ถูกบันทึกในฐานข้อมูล

คำตอบ: เพราะ Customer หรือ Product ยังไม่ไว้ใน database มี Foreign key constraint Error

11. ในคลาส OrderLine จะเกิดอะไรขึ้นหากลืมใส่คำสั่ง @ManyToOne และ @JoinColumn(name = "order_id")
เมื่อประกาศตัวแปร Order

คำตอบ: Hibernate จะไม่รู้ความสัมพันธ์กับ order และจะไม่สร้าง ตาราง order_id

12. คำสั่งในไฟล์ application.properties ใดใช้ควบคุมการสร้างตารางอัตโนมัติ
และเขียนบรรทัดคำสั่งที่ถูกต้องเพื่อเปิดใช้งานการอัปเดต schema อัตโนมัติใน MySQL

คำตอบ: Spring.jpa.hibernate.ddl-auto = update

ส่วนที่ 4: การออกแบบและประยุกต์ใช้งาน (20 คะแนน)

13. กรณีต้องการค้นหาโดยรหัสของลูกค้า จงเขียนฟังก์ชันของ RestController เพื่อแสดงผลลัพธ์ของใบคำสั่งซื้อทั้งหมดของลูกค้ารายนั้น
พร้อมระบุ url ที่ผู้ใช้ต้องป้อนเพื่อให้ผลลัพธ์ดังกล่าว

คำตอบ: Url /api/orders/customer/{id}

```
public List<Order> getOrderbyCustomerId(@PathVariable Long id) {  
    return orderRepository.findByCustomerId(id);  
}
```

14. สมมติว่าคุณต้องการคำนวณมูลค่ารวมของคำสั่งซื้อ (รวมราคาของทุกบรรทัด) เขียนโค้ด Java สำหรับเมธอดใน Service เพื่อคำนวณจากข้อมูลในเอนทิตี Order

คำตอบ: _____

```
public double calculate total (Order order){  
    return order.getOrderLines()  
        .stream()  
        .mapToDouble(line -> line.getProduct().getPrice() * line.  
                                getQuantity())  
        .sum();  
}
```

